IFT 2035

IFT 2035 – Énoncés et solutions : évaluateurs et structures fonctionnelles

Chaque question est suivie de sa solution. Questions **inventées** dans le style des intras ; types de Psil **reconstruits**.

> ⚠️ Le code des solutions n'a pas été compilé : vérifie-le dans GHCi.

Question 4.1 – Évaluateur (style intra E2021 §4)

Énoncé

⏱️ 20 minutes.

Soit l'évaluateur ci-dessous, écrit en Haskell, pour le langage Psil :

```
        type Var = ...                          type Env = ...
        data Value = Vnum Int                   data Lexp = Lnum Int
                   | Vobj Var Lexp                        | Lvar Var
                                                          | Lobjection Var Lexp     --
(objection () x e)
                                                          | Linvoke Lexp Lexp       -- (f e)
                                                          | Lbind Var Lexp Lexp     -- (bind
((x e1)) e2)
        env_lookup :: Env -> Var -> Value
        env_add    :: Env -> Var -> Value -> Env

        eval env (Lnum n) = n
        eval env (Lvar x) = env_lookup x env
        eval env (Lobjection x e) = Vobj x e
        eval env (Linvoke f actual)
          = case f of
              Vnum n -> error "Un nombre n'est pas une fonction"
              Vobj formal body ->
                eval (env_add env formal (eval actual)) body
```

Où `env_lookup` et `env_add` sont des fonctions qu'on présume prédéfinies, qui permettent respectivement de trouver la valeur d'une variable et d'ajouter une variable dans un environnement.

1. Haskell signale des erreurs de typage : indiquer comment corriger le code.
2. `eval` implémente-t-il l'appel par nom (CBN) ou l'appel par valeur (CBV) ? Justifier.
3. `eval` implémente-t-il la portée lexicale ou la portée dynamique ? Justifier.
4. Indiquer comment changer le code pour obtenir l'autre sorte de portée.

5. Haskell se plaint qu'`eval` n'est pas « exhaustive » : compléter sa définition. On veut que le `Lbind` soit **récursif** : la variable déclarée doit être visible dans sa propre définition.

Conseils :

- **1** : vérifie le type de chaque expression, ligne par ligne (il y a 4 erreurs). Regarde l'**ordre des arguments** de `env_lookup`, et ce que `case f of` analyse.
- **3** : où est la **photo** ? Est-ce que `Vobj` mémorise un environnement ?
- **5** : comment rendre un `let` récursif en Haskell ? Pense à la **paresse**.

Solution

1. Erreurs de type (4)

Ligne	Problème	Correction
`eval env (Lnum n) = n`	`n : Int`, mais `eval` doit rendre une `Value`	`= Vnum n`
`env_lookup x env`	mauvais ordre des arguments : le type est `Env -> Var -> Value`	`env_lookup env x`
`case f of`	`f` est une `Lexp`, mais les motifs sont des `Value` (`Vnum`, `Vobj`)	`case eval env f of`
`eval actual`	il manque l'environnement : `eval` prend 2 arguments	`eval env actual`

2. Valeur ou nom ?

Dans l'**intention** du code, c'est l'appel **par valeur** : l'environnement associe le paramètre à une **`Value`** (le résultat de `eval env actual`), et non à une expression non évaluée. En appel par nom, l'environnement contiendrait l'**expression** (et son environnement), réévaluée à chaque utilisation.

Nuance importante : Haskell est paresseux. `eval env actual` n'est donc pas calculé avant l'appel, mais seulement si le corps utilise le paramètre (et une seule fois). En pratique, c'est de l'appel **par nécessité** (*call by need*). Pour un vrai appel par valeur, il faut forcer l'évaluation avec `seq`.

3. Lexicale ou dynamique ?

Dynamique. `Vobj formal body` ne mémorise **aucun environnement** (pas de « photo »). À l'appel, le corps est évalué dans `env_add env formal …`, où `env` est l'environnement **de l'appel**.

4. Passer à la portée lexicale

On ajoute l'environnement de création (la photo) dans la valeur fonction :

```haskell
        data Value = Vnum Int
                   | Vobj Var Lexp Env                          -- la fermeture garde son
environnement

        eval env (Lobjection x e) = Vobj x e env                -- on prend la photo

        eval env (Linvoke f actual)
          = case eval env f of
              Vnum n -> error "Un nombre n'est pas une fonction"
              Vobj formal body fenv ->
                eval (env_add fenv formal (eval env actual)) body   -- le corps dans la PHOTO
fenv
```

L'argument est toujours évalué dans `env` (celui de l'appel). Seul le **corps** change d'environnement.

5. Compléter avec un `Lbind` récursif

```haskell
        eval env (Lbind x e1 e2) =
          let env' = env_add env x (eval env' e1)    -- e1 voit x : récursif
          in  eval env' e2
```

Pourquoi ça ne boucle pas : grâce à la paresse, `eval env' e1` n'est pas calculé au moment où on construit `env'`. Il ne sera calculé que si quelqu'un cherche `x`. Pour une fonction récursive (`e1` est une `Lobjection`), l'évaluation de `e1` produit une fermeture **sans** chercher `x` tout de suite. `x` ne sera cherché que lors d'un appel, quand `env'` existe déjà.

Question 4.2 – Un évaluateur plein d'erreurs

Énoncé

```haskell
        type Var = String

        data Lexp = Lnum Int
                  | Lvar Var
                  | Linvoke Lexp Lexp                   -- (f e)
                  | Lbind [(Var, Lexp)] Lexp            -- (bind ((x1 e1) ...) corps), DOIT
être récursif
                  | Lobjection [(Var, Lexp)] Var Lexp   -- (objection ((a1 e1) ...) x corps)
                  | Levidence Lexp Var                  -- (evidence o a)
                  | Lif Lexp Lexp Lexp                  -- (if c alors sinon)

        data Value = Vnum Int
                   | Vobj [(Var, Value)] (Env -> Value -> Value)

        type Env = [(Var, Value)]
```

```
 1  eval :: Env -> Lexp -> Value
 2  eval env (Lnum n)               = n
 3  eval env (Lvar x)               = lookup x env
 4  eval env (Linvoke f a)          = case eval env f of
 5                                      Vobj _ fun -> fun env a
 6  eval env (Levidence o a)        = case eval env o of
 7                                      Vobj attrs _ -> lookup a attrs
 8  eval env (Lobjection as x body) =
 9    let attrs = [ (a, e) | (a, e) <- as ]
10    in  Vobj attrs (\denv v -> eval ((x, v) : attrs ++ denv) body)
11  eval env (Lbind decls body)     =
12    let env1 = [ (x, eval env e) | (x, e) <- decls ] ++ env
13    in  eval env1 body
14  eval env (Lif c t e)            = case eval env c of
15                                      Vnum 0 -> eval env t
16                                      _      -> eval env e
```

`lookup :: Eq a => a -> [(a, b)] -> Maybe b` est prédéfinie.

1. **Erreurs de type.** Trouver toutes les erreurs de type (il y en a 5). Pour chacune : la ligne, le type attendu, le type obtenu, et une correction qui ne change pas ce que le code essaie de faire.
2. **Erreur dans `Lif`.** Une fois les types corrigés, que donne `(if 0 1 2)` ? Est-ce le comportement attendu, si `0` signifie « faux » ? Corriger.
3. **Portée.** Une fois les types corrigés, cet évaluateur a-t-il la portée lexicale ou dynamique ? Quelles lignes le montrent ? Le **type `Value`** lui-même le trahit : expliquer comment.
4. **Corriger la portée.** Modifier le type `Value` et les lignes nécessaires pour obtenir la portée lexicale.
5. **Récursivité.** Le `Lbind` doit être récursif. L'est-il ? Que donne `(bind ((f (objection () n (f n)))) …)` avec ce code ? Corriger la ligne 12, et expliquer pourquoi la version corrigée ne boucle pas en Haskell.
6. **Prédire**, avec ta version entièrement corrigée (lexicale, `Lbind` récursif), puis avec la version dynamique :

   ```
   (bind ((n 1000)
          (mk (objection () k (objection () x (+ x k)))))
     (bind ((k 1)) ((mk 5) 2)))
   ```

7. **Valeur ou nom.** Avec ta version corrigée, que donne `((objection () x 1) (/ 1 0))` ? Est-ce un appel par valeur ? Que faut-il ajouter à la ligne 5 pour en faire un vrai ?

Conseils :

- **1** : vérifie le type de chaque côté de chaque `=`, et de chaque argument passé à une fonction. Les endroits suspects : `lookup` (qui rend un `Maybe`), et tout ce qui reçoit une `Lexp` au lieu d'une `Value`.
- **3** : d'où vient l'environnement utilisé pour évaluer `body` ? De la création de l'objet, ou de l'appel ?

Solution

1. Erreurs de type (5)

Ligne	Attendu	Obtenu	Correction
2	`Value`	`Int`	`= Vnum n`
3	`Value`	`Maybe Value`	`case lookup x env of Just v -> v ; Nothing -> error ("inconnue : " ++ x)`
5	`fun :: Env -> Value -> Value`, donc un `Value` en 2e argument	`a :: Lexp`	`fun env (eval env a)`
7	`Value`	`Maybe Value`	`case lookup a attrs of Just v -> v ; Nothing -> error "attribut inconnu"`
9	`attrs :: [(Var, Value)]` (pour `Vobj`, et pour `attrs ++ denv` à la ligne 10)	`[(Var, Lexp)]`	`` `[ (a, eval env e) ``

2. `Lif`

`(if 0 1 2)` donne **1** : la branche `Vnum 0` évalue `t`. Si `0` signifie « faux », il faut évaluer **le sinon** :

```
        eval env (Lif c t e) = case eval env c of
                                 Vnum 0 -> eval env e     -- faux → sinon
                                 _      -> eval env t     -- vrai → alors
```

3. Portée

Dynamique. Ligne 5 : `fun env …` passe l'environnement **de l'appel**. Ligne 10 : le corps est évalué dans `denv`, l'environnement reçu à l'appel. L'`env` de la création n'est jamais utilisé pour le corps.

Le type le trahit : la fonction stockée a le type `Env -> Value -> Value`. Elle **attend un environnement à chaque appel**. Une fermeture lexicale n'en a pas besoin, puisqu'elle a déjà le sien.

4. Passer à la portée lexicale

```
        data Value = Vnum Int
                   | Vobj [(Var, Value)] (Value -> Value)      -- plus d'Env en argument

        -- ligne 5
                                           Vobj _ fun -> fun (eval env a)
        -- lignes 9-10
            let attrs = [ (a, eval env e) | (a, e) <- as ]
            in  Vobj attrs (\v -> eval ((x, v) : attrs ++ env) body)   -- env de la CRÉATION
```

La lambda Haskell `\v -> …` **capture** `env` : c'est la photo.

5. Récursivité

Ligne 12 : `eval env e`, et non `eval env1 e`. Les définitions ne voient **pas** `env1`, donc le `Lbind` n'est **pas** récursif.

Pour `(bind ((f (objection () n (f n)))) …)` :

- **avec ce code (dynamique)**, ça « marche » **par accident** : le corps de `f` est évalué dans l'environnement de l'appel, qui contient `f`. Ici, `(f n)` appelle `f` sans fin → **boucle infinie** ;
- **avec la version lexicale** et ce `Lbind` non récursif, la photo de `f` ne contient pas `f` → **variable inconnue `f`**.

Correction :

```
        12    let env1 = [ (x, eval env1 e) | (x, e) <- decls ] ++ env
```

Ça ne boucle pas grâce à la **paresse** : la liste `env1` est construite sans calculer les `eval env1 e`. Ils ne seront calculés que lorsqu'on cherche la variable. Et évaluer une `Lobjection` ne cherche pas encore `f` : on ne le cherche qu'au moment d'un appel.

6. Prédire

```
(bind ((n 1000)
       (mk (objection () k (objection () x (+ x k)))))
  (bind ((k 1)) ((mk 5) 2)))
```

- **Lexicale : 7.** `(mk 5)` crée l'objet intérieur pendant l'appel, avec la photo `{k ↦ 5, …}`. L'appel avec `2` donne `x = 2`, et `k = 5` (photo) → `2 + 5 = 7`.
- **Dynamique : 3.** Quand on appelle l'objet intérieur avec `2`, l'appel `(mk 5)` est **terminé**, donc `k ↦ 5` est dépilé. Dans l'environnement de l'appel, `k ↦ 1` (du `bind` intérieur) → `2 + 1 = 3`.
- `n 1000` est un **piège** : il n'est jamais utilisé.

7. Valeur ou nom

`((objection () x 1) (/ 1 0))` donne **1** : le corps n'utilise pas `x`, et la paresse fait que `(/ 1 0)` n'est jamais évalué. Ce **n'est pas** un appel par valeur (sinon, on aurait une erreur de division par zéro).

Pour forcer l'évaluation de l'argument :

```
        eval env (Linvoke f a) = case eval env f of
                                   Vobj _ fun -> let v = eval env a
                                                 in  v `seq` fun v
```

⚠️ **Attention :** `seq` n'évalue que **jusqu'au premier constructeur**. Si la primitive `/` est écrite `Vnum (x div y)`, alors `seq` s'arrête au constructeur `Vnum` **sans** calculer la division : l'erreur n'apparaît toujours pas. Il faut alors forcer aussi l'entier :

```
                                Vobj _ fun -> let v = eval env a
                                              in  case v of
                                                    Vnum n -> n `seq` fun v
                                                    _      -> fun v
```

ou rendre le champ strict dans le type : `data Value = Vnum !Int | …`. (C'est le piège annoncé dans le thème 6, D5 : « `seq` ne calcule que jusqu'au premier constructeur ; tester avec C6 ».)

Question 4.3 – Évaluateur Psil à compléter

Énoncé

```
        type Var = String

        data Lexp = Lnum Int                        -- 42
                  | Lvar Var                        -- x
                  | Linvoke Lexp Lexp               -- (f e) : appel avec un argument
                  | Lbind [(Var, Lexp)] Lexp        -- (bind ((x1 e1) ... (xn en)) corps) :
RÉCURSIF
                  | Lobjection [(Var, Lexp)] Var Lexp
                       -- (objection ((a1 e1) ... (an en)) x corps) :
                       -- une fonction de paramètre x, avec des attributs a1 ... an
                  | Levidence Lexp Var              -- (evidence o a) : lit l'attribut a de
l'objet o
                  | Lif Lexp Lexp Lexp

        data Value = Vnum Int
                   | Vbool Bool
                   | Vobj [(Var, Value)] (Value -> Value)
                       -- un objet : ses attributs, et la fonction à appeler

        type Env = [(Var, Value)]
```

```
eval :: Env -> Lexp -> Value
eval env (Lnum n)               = Vnum n
eval env (Lvar x)               = case lookup x env of
                                    Just v  -> v
                                    Nothing -> error ("variable inconnue : " ++ x)
eval env (Linvoke f a)          = case eval env f of
                                    Vobj _ fun -> ???
                                    _          -> error "pas une fonction"
eval env (Levidence o a)        = ???
eval env (Lobjection as x body) =
  let attrs = [ (a, eval env e) | (a, e) <- as ]
  in  Vobj attrs (\v -> eval ((x, v) : attrs ++ env) body)
eval env (Lbind decls body)     =
  let env1 = [ (x, eval env1 e) | (x, e) <- decls ] ++ env
  in  eval env1 body
eval env (Lif c t e)            = ???
```

1. Écrire la `Lexp` qui correspond à `(bind ((x 1) (o (objection ((x 10)) y (+ x y)))) (o 5))`. On suppose que `+` est une variable de l'environnement initial, liée à un objet sans attribut.
2. Compléter `Linvoke`, `Levidence` et `Lif`. Pour `Lif`, tout ce qui n'est pas `Vnum 0` ni `Vbool False` est vrai.
3. Dans l'équation `Lobjection` :
   - (a) dans quel environnement sont évalués les attributs ?
   - (b) un attribut peut-il voir le paramètre `x` ? Un autre attribut ?
   - (c) le corps peut-il voir les attributs ?
   - (d) si un attribut et le paramètre ont le même nom, lequel gagne ? Quelle partie du code le décide ?
4. Prédire la valeur de :
   - (a) `(bind ((x 1) (o (objection ((x 10)) y (+ x y)))) (o 5))`
   - (b) `(bind ((o (objection ((x 10)) x x))) (o 5))`
   - (c) `(bind ((a 100)) ((objection ((a 1) (b a)) x b) 0))`
5. **Récursivité.** Dans l'équation `Lbind`, la définition de `env1` utilise `env1` elle-même. Pourquoi est-ce que ça ne boucle pas en Haskell ? Que se passerait-il dans un langage strict ?
6. **Portée.** Cet évaluateur a-t-il la portée lexicale ou dynamique ? Quelle ligne le décide ? Que faudrait-il changer (dans `Value`, `Lobjection` et `Linvoke`) pour avoir la portée dynamique ?
7. **Valeur ou nom.** Prédire `((objection () x 1) (/ 1 0))`. Le résultat correspond-il à l'appel par valeur ? Modifier `Linvoke` pour forcer un vrai appel par valeur (indice : `seq`).

Solution

1. La `Lexp`

```
Lbind [ ("x", Lnum 1)
      , ("o", Lobjection [("x", Lnum 10)] "y"
                (Linvoke (Linvoke (Lvar "+") (Lvar "x")) (Lvar "y")))
      ]
      (Linvoke (Lvar "o") (Lnum 5))
```

`(+ x y)` est un appel curryfié : `+` reçoit `x`, puis le résultat reçoit `y`.

2. Compléter

```
        eval env (Linvoke f a)   = case eval env f of
                                     Vobj _ fun -> fun (eval env a)
                                     _          -> error "pas une fonction"

        eval env (Levidence o a) = case eval env o of
                                     Vobj attrs _ -> case lookup a attrs of
                                                       Just v  -> v
                                                       Nothing -> error ("attribut inconnu : "
++ a)
                                     _            -> error "pas un objet"

        eval env (Lif c t e)     = case eval env c of
                                     Vnum 0      -> eval env e
                                     Vbool False -> eval env e
                                     _           -> eval env t
```

3. L'équation `Lobjection`

- **(a)** Dans `env`, l'environnement **de la création** de l'objet.
- **(b) Non** et **non** : la liste en compréhension utilise `env`, qui ne contient ni `x` ni les autres attributs.
- **(c) Oui** : le corps est évalué dans `(x, v) : attrs ++ env`, qui contient les attributs.
- **(d)** Le **paramètre** gagne : `(x, v)` est placé **avant** `attrs` dans la liste, et `lookup` rend la **première** occurrence.

4. Prédire

- **(a) 15.** Corps : `(y ↦ 5) : [(x ↦ 10)] ++ env`. Le `x` trouvé en premier est l'attribut `10` (il masque le `x ↦ 1` du `bind`) → `10 + 5 = 15`.
- **(b) 5.** Le paramètre `x ↦ 5` est **avant** l'attribut `x ↦ 10` → `5`.
- **(c) 100.** L'attribut `b` est évalué dans l'environnement **de création**, où `a ↦ 100` (du `bind`), et **pas** dans les autres attributs (règle 3b) → `b = 100`. Le corps rend `b` → `100`.

5. Récursivité

Grâce à la **paresse** : `env1` est une liste dont les valeurs (`eval env1 e`) ne sont pas encore calculées. Construire la liste ne demande pas de calculer ces valeurs. Elles le seront seulement quand on cherche une variable, et à ce moment-là `env1` existe déjà.

Dans un langage **strict**, il faudrait calculer `eval env1 e` **avant** de construire `env1`… qui n'existe pas encore : **boucle infinie** (ou impossible). On utiliserait alors une **mutation** : créer l'environnement avec des cases vides, puis les remplir (*backpatching*).

6. Portée

Lexicale. C'est l'équation `Lobjection` qui le décide : la lambda `\v -> eval (… ++ env) body` **capture** `env`, l'environnement de création (la photo). `Linvoke` ne passe que la valeur de l'argument.

Pour la portée **dynamique** :

```
data Value = … | Vobj [(Var, Value)] (Env -> Value -> Value)

eval env (Lobjection as x body) =
  let attrs = [ (a, eval env e) | (a, e) <- as ]
  in  Vobj attrs (\denv v -> eval ((x, v) : attrs ++ denv) body)

eval env (Linvoke f a) = case eval env f of
                           Vobj _ fun -> fun env (eval env a)
```

(Il faut aussi adapter les primitives comme +, pour qu'elles acceptent et ignorent l'`Env`.)

7. Valeur ou nom

`((objection () x 1) (/ 1 0))` donne **1** : la paresse fait que l'argument n'est jamais évalué. Ce n'est **pas** un appel par valeur. Correction : voir la question 4.2 (7), avec `seq`, **et** l'avertissement sur le premier constructeur.

Question 5.1 – File à deux listes

Énoncé

```
data Queue a = Queue [a] [a]
```

- la première liste contient les éléments **à sortir**, dans l'ordre de sortie ;
- la seconde liste contient les éléments **ajoutés récemment**, dans l'ordre **inverse** de leur arrivée.

Par exemple, la file qui contient 1, 2, 3, 4 (1 sortira en premier) peut s'écrire `Queue [1, 2] [4, 3]`.

```
emptyQ = Queue [] []
```

1. Donner le type de `emptyQ`.
2. Donner des types raisonnables pour les fonctions suivantes (totales) :
   - (a) `enqueue` : ajoute un élément à la fin de la file ;
   - (b) `dequeue` : enlève le premier élément de la file, et le renvoie avec la nouvelle file ;
   - (c) `isEmpty` : indique si la file est vide.
3. Écrire le code de `enqueue` et `dequeue`. *Indice : que faire si la première liste est vide, mais pas la seconde ?*
4. À partir de `emptyQ`, on fait : `enqueue 1`, `enqueue 2`, `enqueue 3`, puis `dequeue`, puis `enqueue 4`, puis `dequeue`. Écrire la valeur de la file (`Queue … …`) après chaque opération.
5. Indiquer l'ordre de grandeur du nombre d'opérations en temps CPU et en allocations mémoire pour `enqueue` et pour `dequeue` :
   - (a) dans le pire cas ;
   - (b) en moyenne sur une longue suite d'opérations.
6. *(Bonus)* Pourquoi n'utilise-t-on pas une seule liste, en ajoutant à la fin avec `xs ++ [x]` ?

Solution

1. et 2. Types

```haskell
emptyQ  :: Queue a
enqueue :: a -> Queue a -> Queue a
dequeue :: Queue a -> Maybe (a, Queue a)     -- Maybe : la file peut être vide
isEmpty :: Queue a -> Bool
```

3. Code

```haskell
enqueue x (Queue f b) = Queue f (x : b)              -- O(1) : devant la liste arrière

dequeue (Queue []      []) = Nothing                 -- file vide
dequeue (Queue []      b ) = dequeue (Queue (reverse b) [])   -- on retourne l'arrière
dequeue (Queue (x : f) b ) = Just (x, Queue f b)

isEmpty (Queue [] []) = True
isEmpty _             = False
```

4. Trace

Opération	File	Rendu
`emptyQ`	`Queue [] []`	
`enqueue 1`	`Queue [] [1]`	
`enqueue 2`	`Queue [] [2, 1]`	
`enqueue 3`	`Queue [] [3, 2, 1]`	
`dequeue`	avant vide → `reverse` → `Queue [1, 2, 3] []` → `Queue [2, 3] []`	1
`enqueue 4`	`Queue [2, 3] [4]`	
`dequeue`	`Queue [3] [4]`	2

5. Coût

	Pire cas	En moyenne (amorti)
enqueue	O(1) temps, 1 cellule allouée	O(1)
dequeue	O(n) temps et n cellules allouées (`reverse`)	**O(1)**

Pourquoi O(1) en moyenne : chaque élément est retourné **une seule fois** par `reverse`, quand il passe de l'arrière à l'avant. Sur n opérations, le travail total est O(n).

Nuance : l'argument amorti suppose qu'on n'utilise **pas** plusieurs fois une **ancienne** version de la file. Si on rappelle `dequeue` sur la même vieille file (dont l'avant est vide) plusieurs fois, on refait le `reverse` à chaque fois.

6. Bonus

`xs ++ [x]` **recopie** toute la liste `xs` : O(n) par ajout, donc O(n²) pour n ajouts. Les deux listes donnent O(1) amorti.

Question 5.2 – Zipper

Énoncé

```
        data Zipper a = Zipper [a] a [a]
```

- la première liste contient les éléments **à gauche** du curseur, **à l'envers** (le plus proche en premier) ;
- le champ du milieu est l'élément **sous** le curseur ;
- la dernière liste contient les éléments **à droite** du curseur, dans l'ordre.

Par exemple, la liste `[1, 2, 3, 4, 5]` avec le curseur sur `3` s'écrit `Zipper [2, 1] 3 [4, 5]`.

1. Donner des types raisonnables pour les fonctions suivantes (totales) :
    - (a) `fromList` : crée un zipper à partir d'une liste, curseur sur le premier élément ;
    - (b) `left` / `right` : déplacent le curseur d'une position ;
    - (c) `insert` : insère un élément à gauche du curseur ;
    - (d) `toList` : rend la liste complète, dans l'ordre.
2. Écrire `fromList`, `right` et `toList`.
3. Que doit faire `right` quand le curseur est déjà tout à droite ?
4. Coût en temps et en allocations de `right`, `insert` et `toList`, pour une liste de N éléments.

Solution

1. Types

```
        fromList :: [a] -> Maybe (Zipper a)      -- une liste vide n'a pas d'élément sous le
curseur
        left     :: Zipper a -> Zipper a         -- au bord : on reste sur place
        right    :: Zipper a -> Zipper a
        insert   :: a -> Zipper a -> Zipper a
        toList   :: Zipper a -> [a]
```

(On peut aussi choisir `left`, `right :: Zipper a -> Maybe (Zipper a)`, et rendre `Nothing` au bord. Les deux choix sont raisonnables s'ils sont justifiés.)

2. Code

```haskell
fromList []       = Nothing
fromList (x : xs) = Just (Zipper [] x xs)

right (Zipper ls c (r : rs)) = Zipper (c : ls) r rs
right z                      = z                      -- déjà tout à droite

toList (Zipper ls c rs) = reverse ls ++ (c : rs)
```

3. Au bord droit

Avec le type `Zipper a -> Zipper a`, on **reste sur place** (le zipper est inchangé). Avec `Maybe`, on rend `Nothing`.

4. Coût

	Temps	Allocations
`right`	O(1)	1 cellule + 1 `Zipper`
`insert x (Zipper ls c rs) = Zipper (x : ls) c rs`	O(1)	1 cellule + 1 `Zipper`
`toList`	O(N)	O(N) : `reverse ls` et `++` recopient la partie gauche

Question 5.3 – S-expressions

Énoncé

```haskell
data Sexp = Snil              -- la liste vide ()
          | Ssym String       -- un symbole, par ex. foo
          | Snum Int          -- un nombre, par ex. 42
          | Scons Sexp Sexp   -- une paire : (tête . queue)
```

Une **liste** Lisp est une suite de `Scons` qui se termine par `Snil`. Par exemple, `(+ 1 2)` s'écrit :

```haskell
Scons (Ssym "+") (Scons (Snum 1) (Scons (Snum 2) Snil))
```

(On utilise ici l'ordre habituel de Lisp, tête puis queue. Ce n'est peut-être pas l'ordre de ton TP1.)

1. Écrire la valeur `Sexp` qui représente : (a) `(f x)` ; (b) `(a (b c) d)` ; (c) `((1 2) 3)`.
2. Donner des types raisonnables pour les fonctions suivantes (totales) :
   - (a) `sexpLength` : la longueur d'une liste Lisp ;
   - (b) `sexpToList` : convertit une liste Lisp en liste Haskell de `Sexp` ;
   - (c) `sexpNth` : renvoie le n-ième élément d'une liste Lisp ;
   - (d) `countSyms` : compte tous les symboles d'une `Sexp`, à toutes les profondeurs.

3. Une `Sexp` n'est pas toujours une liste bien formée. Donner un exemple de `Sexp` qui n'est pas une liste, et dire ce que doit renvoyer `sexpToList` dans ce cas.
4. Écrire le code de `sexpToList` et de `countSyms`. Vérifier `countSyms` sur `(a (b c) d)` : le résultat doit être 4.
5. Indiquer l'ordre de grandeur en temps CPU et en allocations mémoire pour :
    - (a) `sexpNth n` sur une liste de longueur N ;
    - (b) `countSyms` sur une `Sexp` contenant N `Scons` au total ;
    - (c) ajouter un élément au début d'une liste Lisp, puis à la fin.

Solution

1. Valeurs

```haskell
-- (a) (f x)
Scons (Ssym "f") (Scons (Ssym "x") Snil)

-- (b) (a (b c) d)
Scons (Ssym "a")
      (Scons (Scons (Ssym "b") (Scons (Ssym "c") Snil))
             (Scons (Ssym "d") Snil))

-- (c) ((1 2) 3)
Scons (Scons (Snum 1) (Scons (Snum 2) Snil))
      (Scons (Snum 3) Snil)
```

2. Types

```haskell
sexpLength :: Sexp -> Maybe Int        -- Nothing si ce n'est pas une liste
sexpToList :: Sexp -> Maybe [Sexp]
sexpNth    :: Int -> Sexp -> Maybe Sexp
countSyms  :: Sexp -> Int              -- toujours défini
```

3. Une `Sexp` qui n'est pas une liste

`Scons (Snum 1) (Snum 2)`, c'est-à-dire `(1 . 2)` : la queue ne finit pas par `Snil`. Ou simplement `Ssym "a"`. `sexpToList` doit renvoyer **`Nothing`**.

4. Code

```haskell
sexpToList :: Sexp -> Maybe [Sexp]
sexpToList Snil        = Just []
sexpToList (Scons h t) = case sexpToList t of
                           Just xs -> Just (h : xs)
                           Nothing -> Nothing
sexpToList _           = Nothing

countSyms :: Sexp -> Int
countSyms (Ssym _)    = 1
countSyms (Scons a b) = countSyms a + countSyms b
countSyms _           = 0
```

Vérification sur `(a (b c) d)` : a, b, c, d → **4** ✅

5. Coût

	Temps	Allocations
(a) `sexpNth n`	O(n)	presque rien (le `Just` final)
(b) `countSyms`	O(N)	rien
(c) ajouter au début	O(1)	1 `Scons`
(c) ajouter à la fin	O(N)	N `Scons` : toute la liste est recopiée

---

Question 5.4 – S-expressions et arbre binaire de recherche

Énoncé

```haskell
data TreeMap b = Empty | Node Int b (TreeMap b) (TreeMap b)

data Sexp = Snil | Ssym String | Snum Int | Scons Sexp Sexp
```

On veut **sauvegarder** un arbre dans un fichier, sous forme de S-expression, puis le **relire**. Encodage :

- l'arbre vide `Empty` est représenté par `()`, c'est-à-dire `Snil` ;
- un nœud `Node k v g d` est représenté par la liste de 4 éléments `(k v g' d')`, où `g'` et `d'` sont les encodages des sous-arbres.

Par exemple, `Node 5 (Ssym "x") Empty Empty` est encodé par `(5 x () ())`.

1. Écrire, en notation Lisp, l'encodage de :

```haskell
t = Node 50 (Ssym "e") (Node 30 (Ssym "c") Empty Empty)
                       (Node 70 (Ssym "g") Empty Empty)
```

2. Donner des types raisonnables pour : (a) `toSexp`, qui encode un `TreeMap Sexp` en `Sexp` ; (b) `fromSexp`, qui décode une `Sexp` en `TreeMap Sexp`. Expliquer pourquoi les deux types ne sont pas symétriques.
3. Donner deux exemples de `Sexp` que `fromSexp` doit refuser, pour deux raisons différentes.
4. Écrire `toSexp`. *Indice : écris d'abord `list4 :: Sexp -> Sexp -> Sexp -> Sexp -> Sexp`.*
5. Écrire `fromSexp`.
6. Écrire `sexpLookup :: Int -> Sexp -> Maybe Sexp`, qui cherche une clé directement dans l'encodage, sans le décoder d'abord.
7. Coût en temps et en allocations de : (a) `toSexp` pour un arbre de N nœuds ; (b) `sexpLookup` comparé à `tmLookup`, pour un arbre de profondeur h. Lequel est le plus rapide, et pourquoi la différence reste-t-elle petite ?

Conseil (5 et 6) : le motif qui reconnaît exactement une liste de 4 éléments est `Scons (Snum k) (Scons v (Scons g (Scons d Snil)))`.

Solution

1. Encodage

```
        (50 e (30 c () ()) (70 g () ()))
```

2. Types

```
        toSexp   :: TreeMap Sexp -> Sexp
        fromSexp :: Sexp -> Maybe (TreeMap Sexp)
```

Pas symétriques : tout arbre peut être encodé, donc `toSexp` est toujours défini. Mais **toute** `Sexp` n'est pas un arbre valide, donc `fromSexp` doit pouvoir échouer : `Maybe`.

3. Deux `Sexp` à refuser

- **Mauvaise longueur :** `(50 e ())`, une liste de 3 éléments au lieu de 4.
- **Clé qui n'est pas un nombre :** `(x e () ())`.
- (Autres possibles : `42` seul, une liste mal terminée, ou un arbre qui ne respecte pas l'ordre des clés, si on veut aussi vérifier la propriété de recherche.)

4. `toSexp`

```
        list4 :: Sexp -> Sexp -> Sexp -> Sexp -> Sexp
        list4 a b c d = Scons a (Scons b (Scons c (Scons d Snil)))

        toSexp :: TreeMap Sexp -> Sexp
        toSexp Empty          = Snil
        toSexp (Node k v g d) = list4 (Snum k) v (toSexp g) (toSexp d)
```

5. `fromSexp`

```haskell
fromSexp :: Sexp -> Maybe (TreeMap Sexp)
fromSexp Snil = Just Empty
fromSexp (Scons (Snum k) (Scons v (Scons g (Scons d Snil)))) =
  case (fromSexp g, fromSexp d) of
    (Just g', Just d') -> Just (Node k v g' d')
    _                  -> Nothing
fromSexp _ = Nothing
```

6. `sexpLookup`

```haskell
sexpLookup :: Int -> Sexp -> Maybe Sexp
sexpLookup _ Snil = Nothing
sexpLookup k (Scons (Snum k') (Scons v (Scons g (Scons d Snil))))
  | k == k'   = Just v
  | k <  k'   = sexpLookup k g
  | otherwise = sexpLookup k d
sexpLookup _ _ = Nothing          -- encodage invalide
```

7. Coût

- **(a)** **`toSexp`** : O(N) en temps. Par nœud : 4 `Scons` + 1 `Snum`, soit environ **5N** allocations.
- **(b)** **`sexpLookup` contre `tmLookup`** : les deux sont **O(h)** et n'allouent presque rien. `tmLookup` est un peu plus rapide : à chaque niveau, il lit **un** `Node`, alors que `sexpLookup` doit traverser **4** `Scons` pour atteindre les sous-arbres. La différence reste petite, parce que c'est un **facteur constant** : le nombre de niveaux visités est le même.

Erreurs fréquentes à éviter

1. Oublier d'envelopper avec `Vnum` ou `Just` (types).
2. `lookup` rend un `Maybe` : il faut toujours l'ouvrir avec `case`.
3. Passer une `Lexp` là où on attend une `Value` : il manque un `eval env …`.
4. Confondre l'environnement de la **création** (lexical, la photo) et celui de **l'appel** (dynamique).
5. Croire que `seq` force tout : il s'arrête au **premier constructeur**.
6. `let` récursif : écrire `eval env e` au lieu de `eval env1 e`.

Checklist avant de rendre

```
□ Types d'abord, code ensuite
□ Fonctions totales : Maybe au lieu de error
□ Un cas par constructeur ; toutes les combinaisons de motifs couvertes
□ Recopier le chemin, partager le reste (coût en allocations)
□ Si l'énoncé donne un exemple, je déroule mon code dessus
□ Évaluateur : dans quel environnement est évalué le corps ? (photo = lexical, appel =
dynamique)
□ Évaluateur : l'argument est-il calculé avant l'appel ? (Haskell est paresseux : seq)
```